Extensions du langage C#



Le langage C# est un langage relativement riche, toutefois on peut encore l'enrichir selon les besoins spécifiques d'un projet ou d'une entreprise...

D'ailleurs, LINQ utilise ce mécanisme pour étendre les possibilités des `IEnumerable`, par exemple si on regarde la signature de `Where` :

```
public static System.Linq.IQueryable<TSource> Where<TSource> (this
System.Linq.IQueryable<TSource> source,
System.Linq.Expressions.Expression<Func<TSource,bool>> predicate);
//                                                              ^
//                                                          C'est quoi ça ???
```

La déclaration d'une méthode d'extension répond à trois règles:

1. Elle est `public static`
2. Elle appartient à une classe qui est également `public static`
3. Son premier paramètre est du type qu'on veut étendre, préfixé par le mot clé `this`

Pour le reste, il s'agit d'une méthode conventionnelle:

- Elle peut retourner une valeur - ou rien
- Elle peut recevoir des paramètres - ou pas

Pour que vos méthodes d'extension soient utilisables, il faut que la classe qui les définit se trouve

- Soit dans le même namespace que votre code
- Soit dans un namespace importé avec `using`

Exemple:

```
public static class PeopleExtensions
{
    public static string Greetings(this string name)
    {
        return $"Hello {name}";
    }
}
```

Ce code étend la classe `string` de C# en ajoutant une méthode `Greetings()`:

```
Console.WriteLine("Bob".Greetings());
```

Ce code affichera

```
Hello Bob
```

Chaînage grâce à la valeur de retour

Quand une méthode d'extension retourne un élément, cela permet de faire du chaînage.
En effet, si la méthode retourne un résultat, ce dernier a un type.
Et si on a défini une méthode d'extension pour ce type, on peut l'utiliser directement sur le résultat.

Exemple 1: chaînage

```
public static void ToConsole(this string subject)
{
    Console.WriteLine(subject);
}
```

"Bob" est une `string`, on peut donc faire `"Bob".Greetings()`
Le résultat de `"Bob".Greetings()` est de type `string`, on peut donc faire `"Bob".Greetings().ToConsole()`

> Cet exemple montre que le chaînage fonctionnel part des données (ici "Bob").
> C'est un point de vue qui peut être déstabilisant, car on avait l'habitude jusque-là de choisir d'abord l'action (avec Console.Write)...

Quel sera le résultat de `"Bob".Greetings().Greetings().ToConsole()` ?

▶ réponse

Quel sera le résultat de `"Bob".Greetings().ToConsole().Greetings()` ?

▶ réponse

Exemple 2: avec paramètres

```
    /// <summary>
    /// Passe en minuscule les éléments fournis, éventuellement de manière
aléatoire
    /// </summary>
    /// <param name="subject">Les données en input</param>
    /// <param name="random">Si vrai, aura 50% de faire la transformation</param>
    /// <returns></returns>
    public static IEnumerable<string> ToLower(this IEnumerable<string>
subject,bool random=false)
    {
        return subject.Select(text =>
            random?
                randomGenerator.Next(2)==1?
                    text.ToLower():text
                :text.ToLower());
    }
```

> On constate dans cet exemple qu'une méthode d'extension peut avoir des paramètres standards à la suite du premier (ici bool random=false)...

Quel sera le résultat du code suivant :

```
var data = new[] {"BoB","Max","jOelLe","NadiA" };
data
    .Where(name => name.StartsWith("j"))
    .ToLower(true)
    .ToList()
    .ForEach(Console.WriteLine);
```

▶ Réponse

DSL : *D*omain *S*pecific *L*anguage

Un exemple concret de l'utilisation des extensions est de pouvoir créer un `langage spécifique au domaine`. On entend par là un pseudo-langage qui est plus proche du métier.

Exemple 1 : FluentAssertions

Ceci a été utilisé pour la librairie d'assertion pour les tests `FluentAssertions`, qui permet d'écrire les assertions ainsi :

```
[Fact]
public void TestIsMatch()
{
    //Arrange
    var cmd = "--help";
```

```
    //Act
    bool result = cmd.IsMatch(Program.OptHelp);

    //Assert
    result.Should().BeTrue("la ligne de commande contient le paramètre --help");
}
```

> `result.Should().BeTrue` se lit et se comprend facilement (le résultat devrait retourner 'vrai')

Exemple 2 : Cosmos

Pour le projet cosmos, un DSL a été mis en place pour facilement tester certains aspects du langage :

```
[Fact]
public void TestDifferentNumber()
{
    TestBoolean("5".IsDifferentThan("6"), true);
}
```

> Sans connaître le langage Cosmos, on comprend que ce teste vérifie la différence entre deux valeurs...
> Derrière les décors, IsDifferentThan convertit cela en langage Cosmos...

Méthodes standards vs Extensions

En réalité, les extensions n'offrent rien de plus que les méthodes standards, si ce n'est le fait qu'elles peuvent contribuer à rendre le code lisible de manière plus naturelle.

Exemple 1: Normalisation de chaîne de caractères

```
// Méthodes standards
string input = Console.ReadLine();
string normalizedInput = ToUpperCase(RemoveSpaces(Trim(input)));

// Avec extensions – même chose, lecture de gauche à droite
string normalizedInput = input.Trim().RemoveSpaces().ToUpperCase();

// Et si la liste d'actions s'allonge
string normalizedInput = input
    .Trim()
    .RemoveSpaces()
    .ReplaceAccents().
    .Justify(right: true)
    .Crop(8)
    .ToUpperCase()
```

Exemple 2: LinQ

LinQ propose principalement des extensions:

- De types générique énumérable
- Qui reçoivent une fonction en paramètre

```
IEnumerable<int> numbers = new List<int> { 13, 32, -23, -4, 15, 22, 98 };
// IEnumerable<T> → Where → IEnumerable<T> → Select → IEnumerable<T> → OrderBy →
...
numbers
    .Where(n => n > 0)          // IEnumerable<int>
    .Select(n => n * 2)         // IEnumerable<int>
    .OrderBy(n => n)            // IOrderedEnumerable<int> (compatible)
    .Take(3)                    // IEnumerable<int>
    .ToList();                  // List<int> — matérialisation
```

Conclusion ⚖

Les extensions du langage sont une bonne manière d'accélérer le développement en enrichissant le langage de base avec ce qu'on a besoin de manière répétitive...
Bien entendu, cela implique d'avoir une dépendance supplémentaire vers ces extensions et cet aspect doit être pris en compte dans la balance au moment de choisir de les utiliser ou pas...